Lab 05

TITLE

“Build, Test, and Package Applications with IntelliJ IDEA and Docker”

Name:
Areeba Hameed

Registration number:
(2023-BSCS-25)

Section:
A

GitHub username:
areebahameed936-dot

Lab 5: Build, Test, and Package Applications with IntelliJ IDEA and Docker

Estimated duration: 3 hours

IntelliJ setup repository: Intellij-Idea-docker

Submission repository: DevOps

Submission directory: Labs/Lab05

Screenshot directory: Labs/Lab05/screenshots

Clone the IntelliJ Docker repository on your local Ubuntu computer and launch its browser desktop. Use IntelliJ's Clone Repository / Get from Version Control action to download four projects, one at a time. Install and run the build tools from the terminal inside IntelliJ IDEA.

```
DevOps/
└── Labs/
    └── Lab05/
        ├── README.md
        ├── Lab5_Solution.pdf
        └── screenshots/
```

The instructor supplies this README. Create your own screenshots directory and solution PDF.

Lab Objective

By the end of this lab, you will be able to:

- Launch IntelliJ IDEA through the supplied Docker environment.
- Clone and open existing projects through IntelliJ.
- Install Maven, add an Elasticsearch dependency, test the project, package it and run its generated JAR.
- Configure the Gradle project and Gradle JVM to use JDK 16, add an Elasticsearch dependency, sync it and rebuild it with the Gradle Wrapper.
- Install Node.js and npm, download dependencies, run tests, create a package with npm pack and start a Node.js application.
- Resolve a port conflict by changing the application port to 4000.
- Build and inspect a bundled Node.js API artifact with webpack.
- Build a Docker image from an existing Dockerfile and identify its image tag.

Prerequisites

- Use the Ubuntu account and Git access prepared in the earlier labs.
- Docker Engine and Docker Compose are already installed on your local computer.
- Your host Ubuntu username matches your registered GitHub username, and the hostname is ubuntu.
- Your computer has a browser and a screenshot tool.
- Allow time and disk space for the first Docker image, IDE, JDK 16 and dependency downloads.

- Use your normal Ubuntu login account to start the Docker environment.
- The repository configures the IDE terminal prompt to display the host user's name with @ubuntu. The desktop's internal account is abc; use the supplied setup and your registered host login for the required prompt.

Where to Run Commands

Location	Commands used there
Ubuntu host terminal	Clone the IntelliJ Docker repository and run setup.sh
IntelliJ's Terminal tool window	Install Maven/Node.js, build and test projects, run the JARs, use npm and run Docker commands

Screenshot Rules

Read these rules before starting. Each required screenshot is evaluated separately.

Exam evaluation screenshots are not graded. They are for practice only and do not count toward the lab marks.

Use an Ubuntu account whose terminal prompt shows your registered GitHub username before @ and ubuntu as the hostname:

```
YourGitHubUsername@ubuntu:~$
```

The comparison is case-insensitive, but the spelling must match the user_name registered in the Student Marks Portal.

Every terminal screenshot must clearly show:

- YourGitHubUsername@ubuntu in the prompt;
- the required command; and
- its relevant result or output.

For IntelliJ dialogs, Welcome screens and browser pages, keep a host Ubuntu terminal or IntelliJ terminal showing YourGitHubUsername@ubuntu visible in the same screenshot. For build and run commands, show IntelliJ's own Terminal tool window with that prompt and the relevant output.

Save every screenshot inside:

```
Labs/Lab05/screenshots/
```

Use the exact filename and capitalization given in this document. Do not add numbers or alternative spellings.

Screenshots may use any image dimensions, but their text and evidence must be clear and readable. Screenshots must be readable by OCR. A missing, unclear, cropped, or unreadable screenshot receives zero for that check.

A screenshot containing only a command, only filled but unsaved form fields, or unrelated output is not proof of completion. The required command and its relevant output must be visible.

Do not expose real passwords, password hashes, private SSH keys, access tokens, recovery codes or other secrets in a screenshot or solution file.

Exact and visually near-duplicate screenshots are compared across students for the same task. If two students submit the same or slightly modified screenshot, both receive zero for that task.

Commit and push the complete Labs/Lab05 directory to your DevOps repository before the deadline. Files that remain only on your computer cannot be graded.

Required Screenshot Filenames

These are the 37 required Lab 5 screenshot filenames:

1. intellij_repo_clone.png	20. gradle_stopped.png
2. intellij_running.png	21. gradle_project_closed.png
3. intellij_desktop.png	22. node_project_clone.png
4. maven_project_clone.png	23. nodejs_installed.png
5. maven_installed.png	24. npm_install.png
6. maven_package.png	25. node_running.png
7. maven_elasticsearch_dependency.png	26. node_browser.png
8. maven_repackage.png	27. npm_test.png
9. maven_test.png	28. npm_pack.png
10. maven_jar.png	29. react_node_project_clone.png
11. maven_running.png	30. react_api_dependencies.png
12. maven_stopped.png	31. react_api_build.png
13. maven_project_closed.png	32. react_api_bundle.png
14. gradle_project_clone.png	33. docker_installed.png
15. gradle_build.png	34. dockerfile_exists.png
16. gradle_elasticsearch_dependency.png	35. docker_images_before.png
17. gradle_rebuild.png	36. docker_build.png
18. gradle_jar.png	37. docker_images_after.png
19. gradle_running.png	

Each required screenshot has equal weight. Exam-practice questions are not included in lab marks. Automated grading requires a configured Lab 5 grader.

Getting Started

Create the screenshot directory in your existing DevOps repository:

```
cd ~/DevOps
mkdir -p Labs/Lab05/screenshots
```

Save the images there and include them in Lab5_Solution.pdf in task order.

Task 1: Clone and Launch IntelliJ IDEA

In your Ubuntu host terminal, clone the supplied repository:

```
cd ~
git clone https://github.com/WaqasSaleem97/Intellij-Idea-docker.git
cd Intellij-Idea-docker
```

Screenshot as `intellij_repo_clone.png`, showing the clone command and successful download.



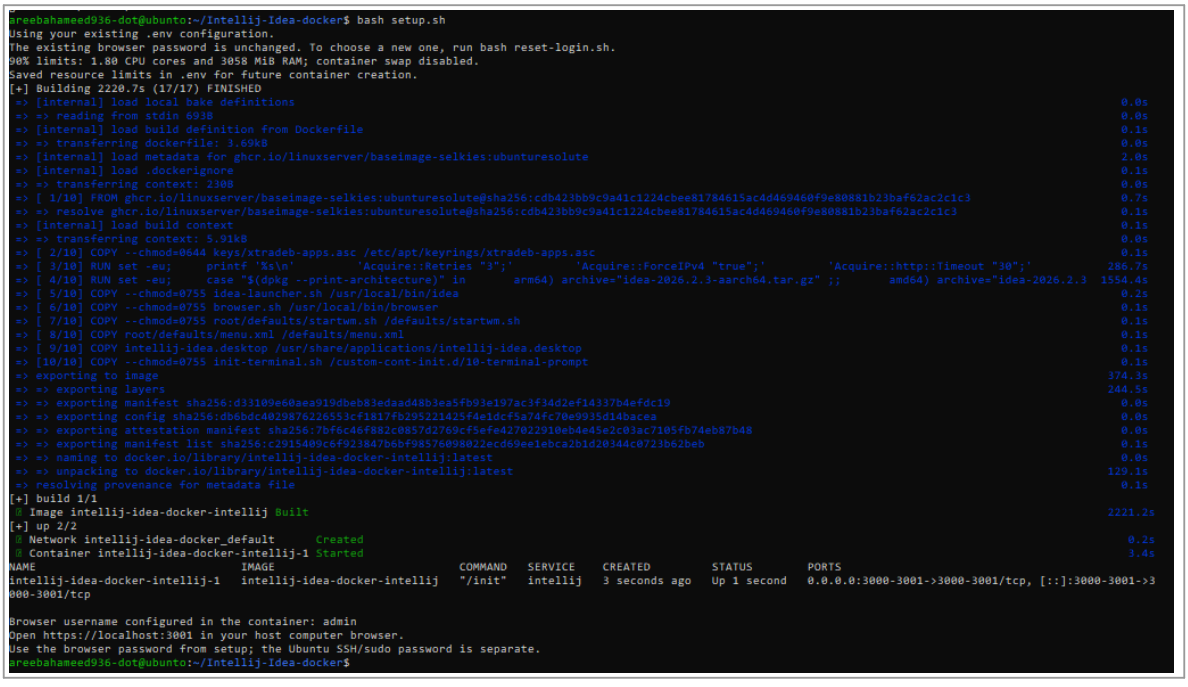
Screenshot 1: intellij_repo_clone.png

Build and launch the environment:

```
bash setup.sh
docker compose ps
```

Enter and confirm the browser password when prompted. The script builds the image and starts it with Docker Compose. The first build downloads the desktop and IDE.

Screenshot as `intellij_running.png`, showing the running intellij service and desktop port 3001. Keep passwords outside the screenshot.



Screenshot 2: intellij_running.png

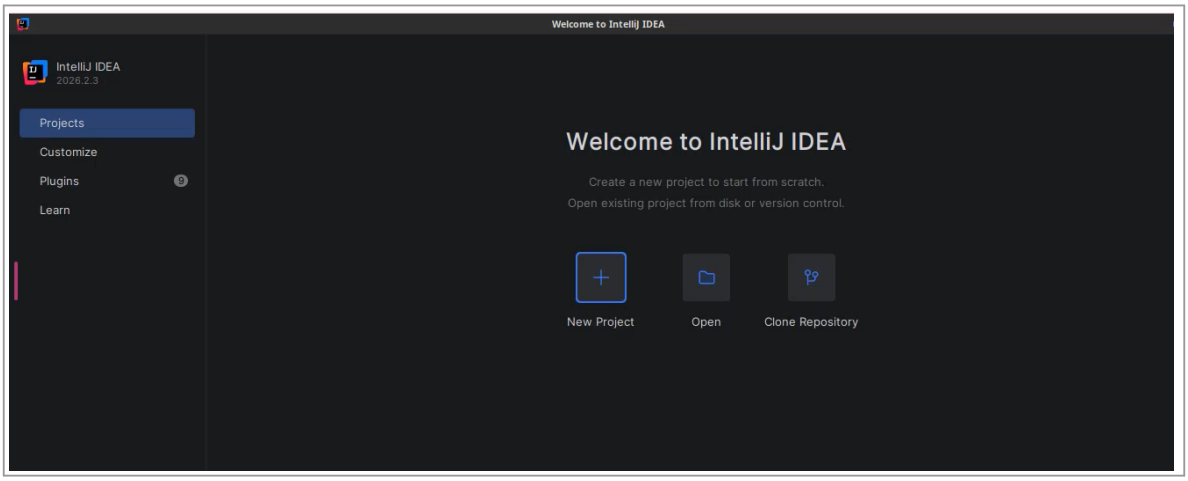
Open https://localhost:3001 in your host computer's browser. Acknowledge the self-signed certificate notice for this local desktop address and sign in with browser username admin and the password you chose during setup.

Launch IntelliJ IDEA from the browser desktop's applications menu. If needed, open the terminal in that browser desktop and run:

```
idea
```

Complete the first-run screens and reach the IntelliJ Welcome screen.

Screenshot as `intellij_desktop.png`, showing IntelliJ in the browser desktop and your Ubuntu host terminal beside it.



Screenshot 3: intellij_desktop.png

Create all four projects under /config/IdeaProjects. This directory is stored in the container's persistent /config volume.

Screenshot Required: `intellij_repo_clone.png`, `intellij_running.png`, `intellij_desktop.png`

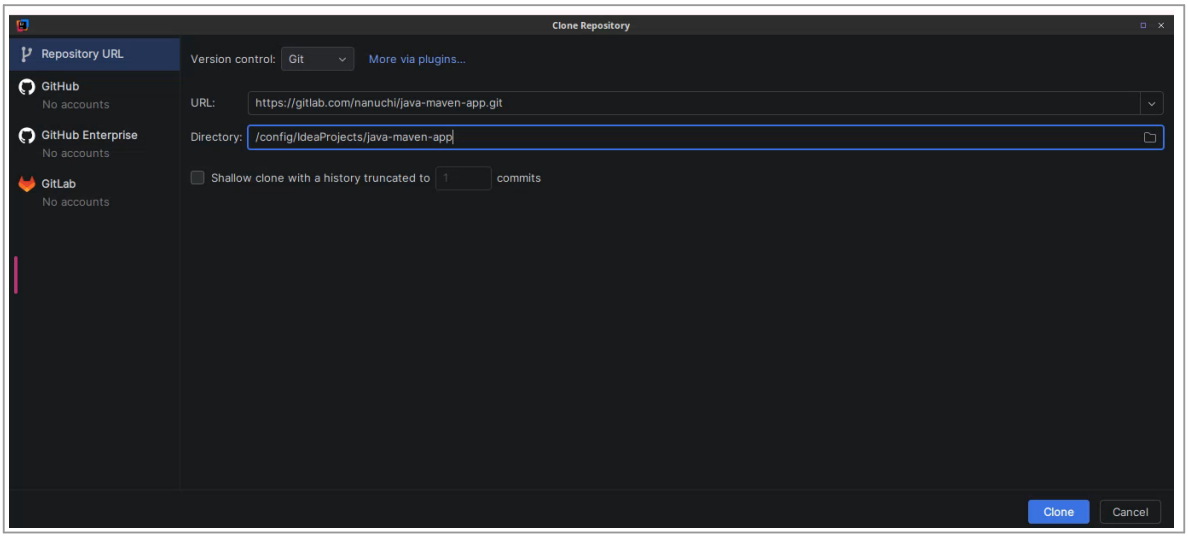
Task 2: Build and Run the Maven Project

On IntelliJ's Welcome screen, select Clone Repository or Get from Version Control. Enter:

URL: https://gitlab.com/nanuchi/java-maven-app.git

Directory: /config/IdeaProjects/java-maven-app

Screenshot as `maven_project_clone.png`, showing the repository URL and destination in the clone dialog.



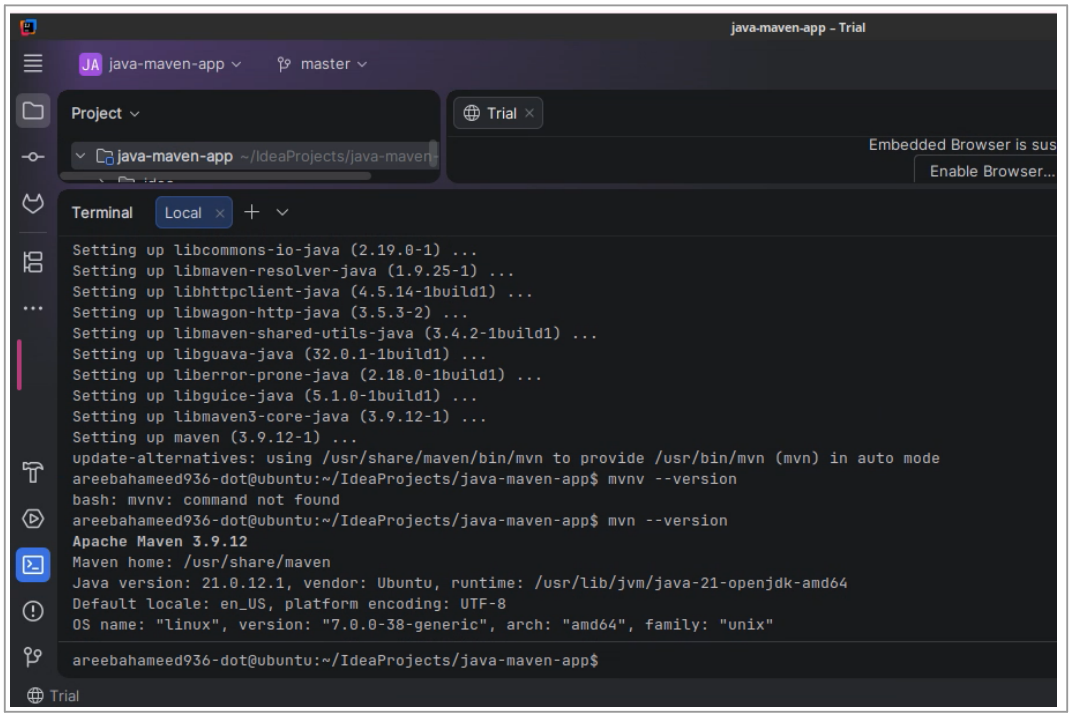
Screenshot 4: maven_project_clone.png

Click Clone, open/trust the course sample and wait for it to load. Use the Maven project configuration when asked. The project contains pom.xml.

Open View → Tool Windows → Terminal inside IntelliJ. Work from the Maven project's root directory. Run:

```
sudo apt update
sudo apt install -y maven
mvn --version
```

Screenshot as `maven_installed.png`, showing the installed Maven version in IntelliJ's terminal.



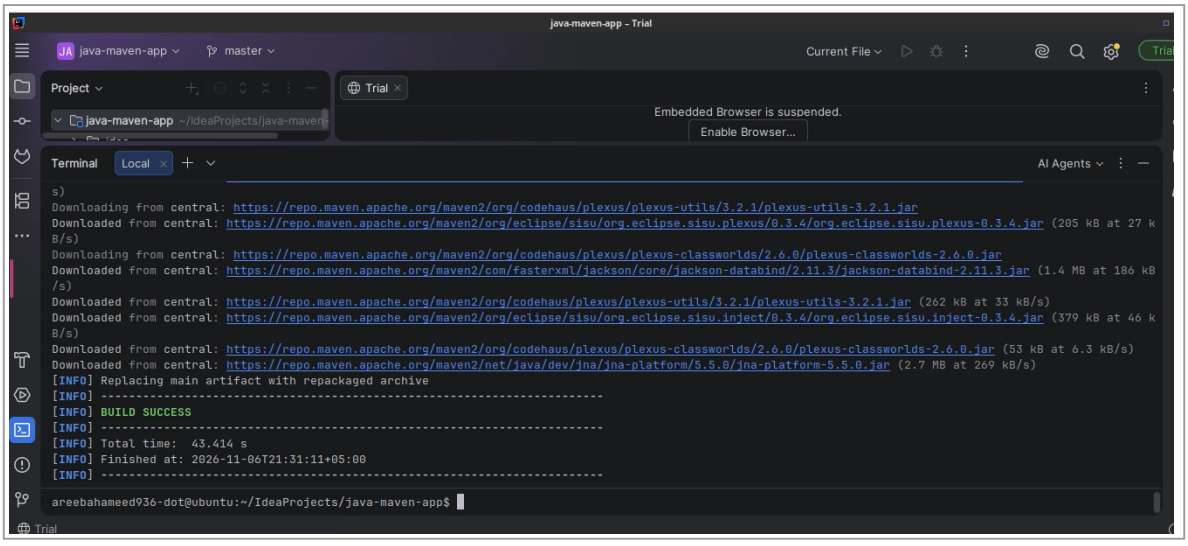
Screenshot 5: maven_installed.png

Package the application:

```
mvn package
```

Wait for BUILD SUCCESS.

Screenshot as `maven_package.png`, showing the command and successful build.



Screenshot 6: maven_package.png

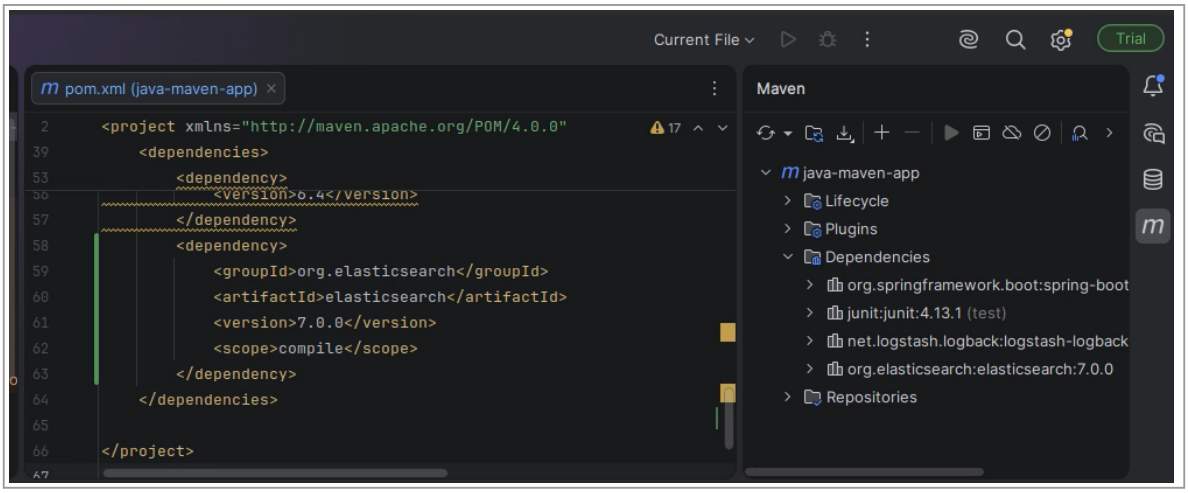
Open pom.xml in IntelliJ. Add this dependency inside the existing <dependencies> section, alongside the other project dependencies:

```
<dependency>
    <groupId>org.elasticsearch</groupId>
    <artifactId>elasticsearch</artifactId>
    <version>7.0.0</version>
    <scope>compile</scope>
```

```
</dependency>
```

Save the file. Click Sync Maven Changes in the editor, or open the Maven tool window and choose Reload All Maven Projects. Wait for synchronization and dependency downloads to finish. Expand Dependencies in the Maven tool window and locate org.elasticsearch:elasticsearch:7.0.0.

Screenshot as `maven_elasticsearch_dependency.png`, showing the dependency in pom.xml and the synchronized Elasticsearch 7.0.0 entry. Keep your required terminal prompt visible.



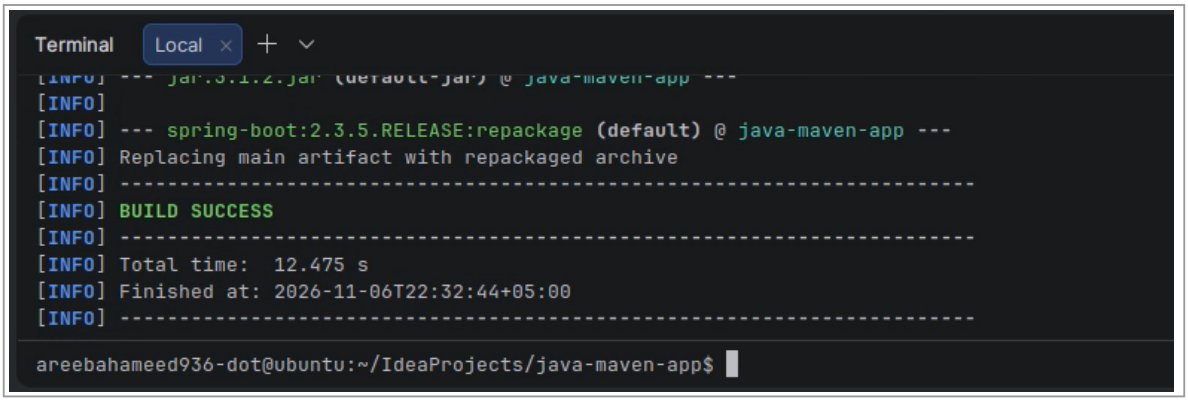
Screenshot 7: maven_elasticsearch_dependency.png

Package the project again after adding the dependency:

```
mvn package
```

Wait for BUILD SUCCESS.

Screenshot as `maven_repackage.png`, showing the command and the successful second package build.

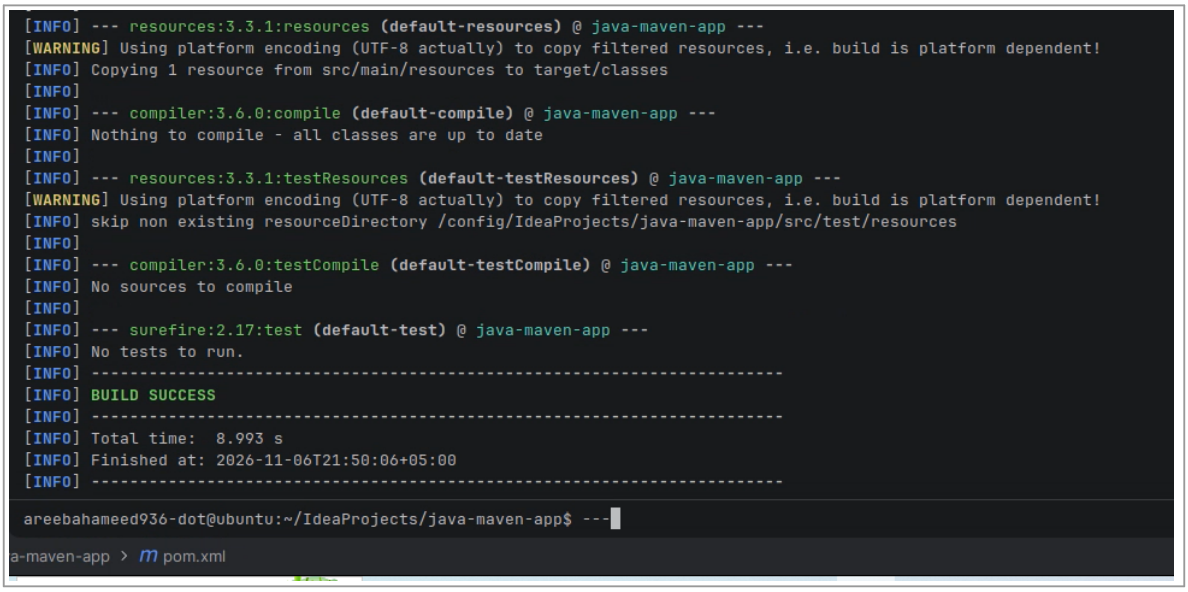


Screenshot 8: maven_repackage.png

Run Maven's test phase:

```
mvn test
```

Screenshot as `maven_test.png`, showing the command, test output and build result.



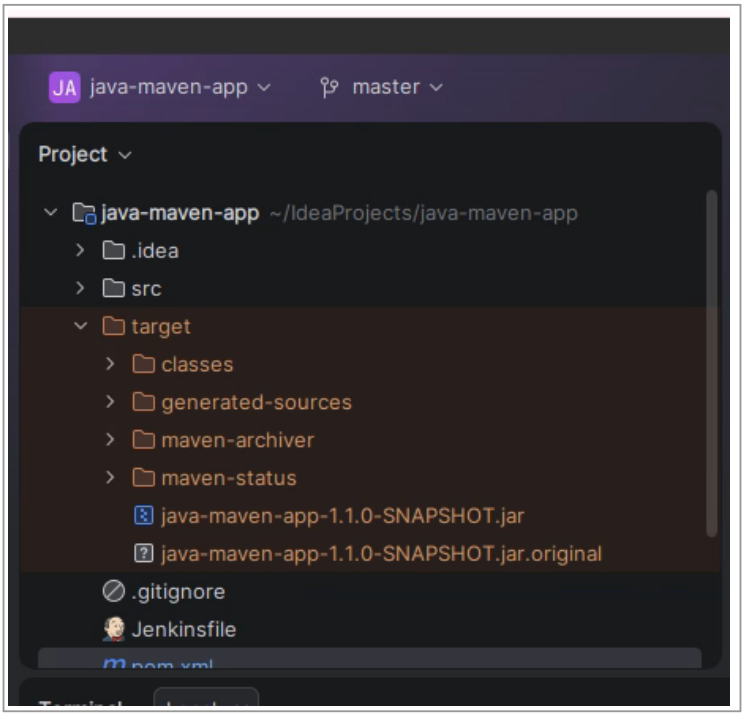
Screenshot 9: maven_test.png

> The current sample has no test source files, so Maven may report No tests to run with BUILD SUCCESS. Record that actual output; it means the test phase ran without finding tests.

In IntelliJ's Project pane, refresh the project if needed and expand target. Locate:

```
target/java-maven-app-1.1.0-SNAPSHOT.jar
```

Screenshot as `maven_jar.png`, showing the target directory and the exact JAR filename.



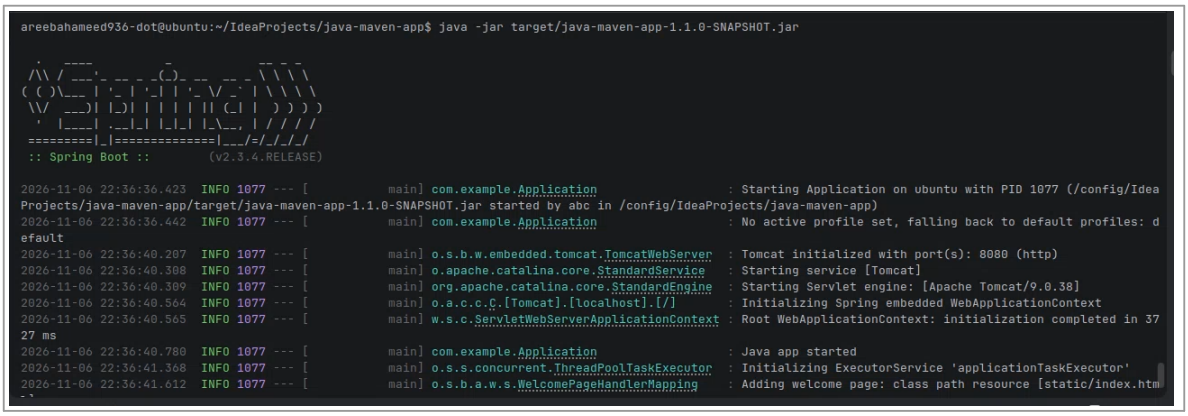
Screenshot 10: maven_jar.png

In the same IntelliJ terminal, run the JAR from the project root:

```
java -jar target/java-maven-app-1.1.0-SNAPSHOT.jar
```

Wait for the Spring Boot/Tomcat startup output. The sample runs as a web application on port 8080.

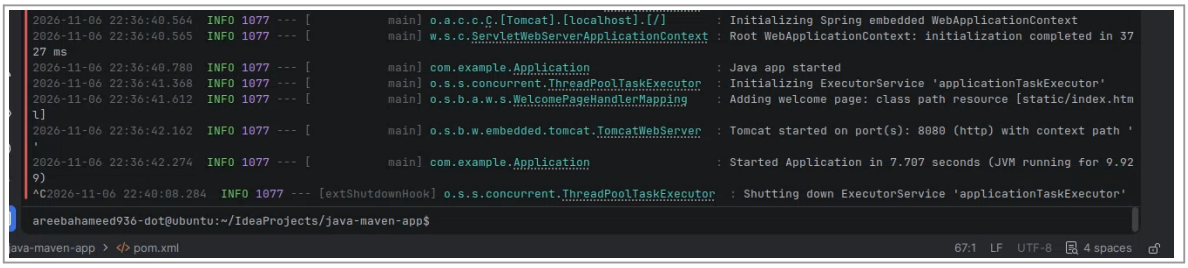
Screenshot as `maven_running.png`, showing the JAR command and successful application startup.



Screenshot 11: maven_running.png

Press Ctrl+C in that terminal to stop the application. Wait for the shell prompt to return.

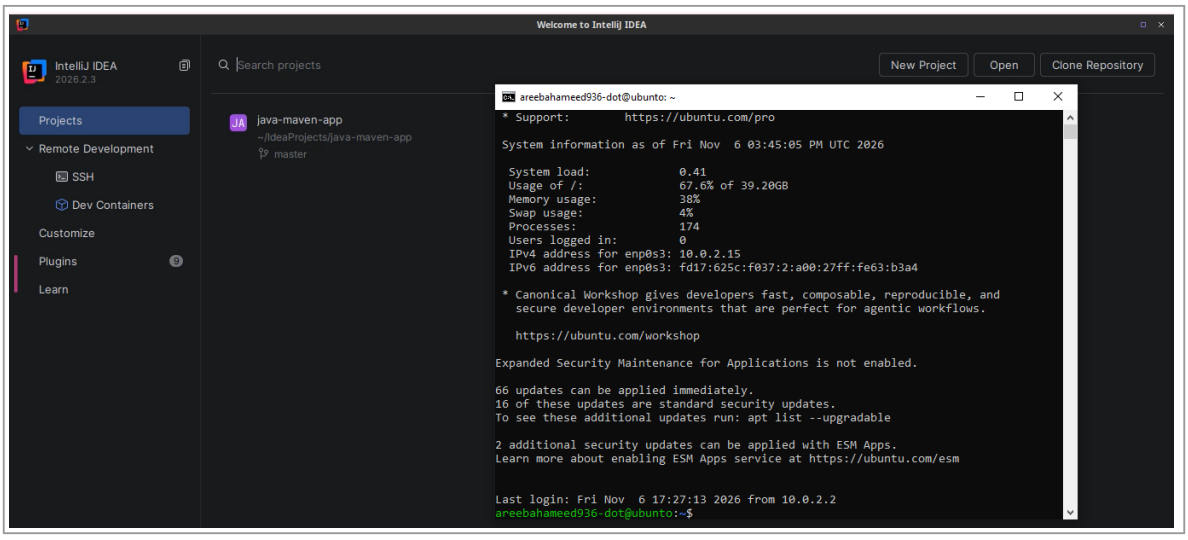
Screenshot as `maven_stopped.png`, showing ^C/shutdown output and the returned prompt.



Screenshot 12: maven_stopped.png

Select File → Close Project and return to IntelliJ's Welcome screen.

Screenshot as `maven_project_closed.png`. Use your Ubuntu host terminal beside the browser for identity evidence.



Screenshot 13: maven_project_closed.png

Screenshot Required: `maven_project_clone.png`, `maven_installed.png`, `maven_package.png`, `maven_elasticsearch_dependency.png`, `maven_repackage.png`, `maven_test.png`, `maven_jar.png`,

`maven_running.png`, `maven_stopped.png`, `maven_project_closed.png`

Summary: You added Elasticsearch 7.0.0 to pom.xml, synchronized the Maven project, packaged it again and ran the test phase. Maven created the executable JAR in target, and Java ran it.

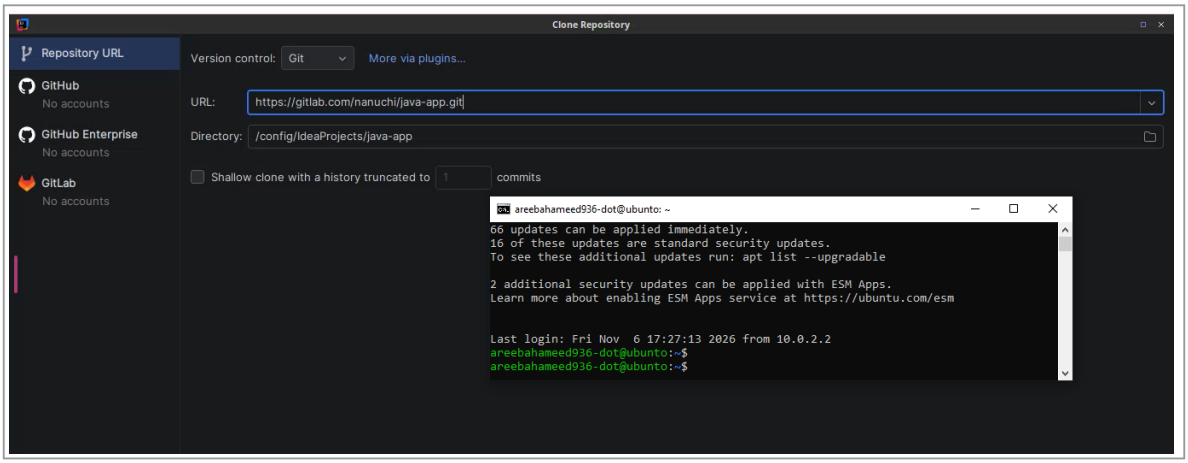
Task 3: Build and Run the Gradle Project

From IntelliJ's Welcome screen, use Clone Repository / Get from Version Control again:

URL: https://gitlab.com/nanuchi/java-app.git

Directory: /config/IdeaProjects/java-app

Screenshot as `gradle_project_clone.png`, showing the URL and destination.



Screenshot 14: gradle_project_clone.png

Click Clone, open/trust the sample and load it as a Gradle project. It contains build.gradle, settings.gradle and gradlew.

Select the project's Gradle Wrapper when IntelliJ asks which Gradle installation to use.

Configure JDK 16 before running the Gradle Wrapper:

- Select File → Project Structure → Project.
- Set Project SDK to JDK 16.
- Set Project language level to 8 - Lambdas, type annotations, etc.
- If JDK 16 is unavailable, open the SDK list and select Download JDK. Use a JDK build and architecture compatible with your computer.

Then open File → Settings → Build, Execution, Deployment → Build Tools → Gradle and set Gradle JVM to Project SDK (JDK 16) or the downloaded JDK 16. Select Apply and OK, then close and reopen IntelliJ's terminal.

The supplied wrapper is Gradle 7.x. It can run on Java 16, but it cannot run on Java 21. Java 21 produces class-file major version 65 and requires Gradle 8.5 or newer to run Gradle itself.

Open View → Tool Windows → Terminal. From the Gradle project root, verify the JVM used by the wrapper:

```
./gradlew --version
```

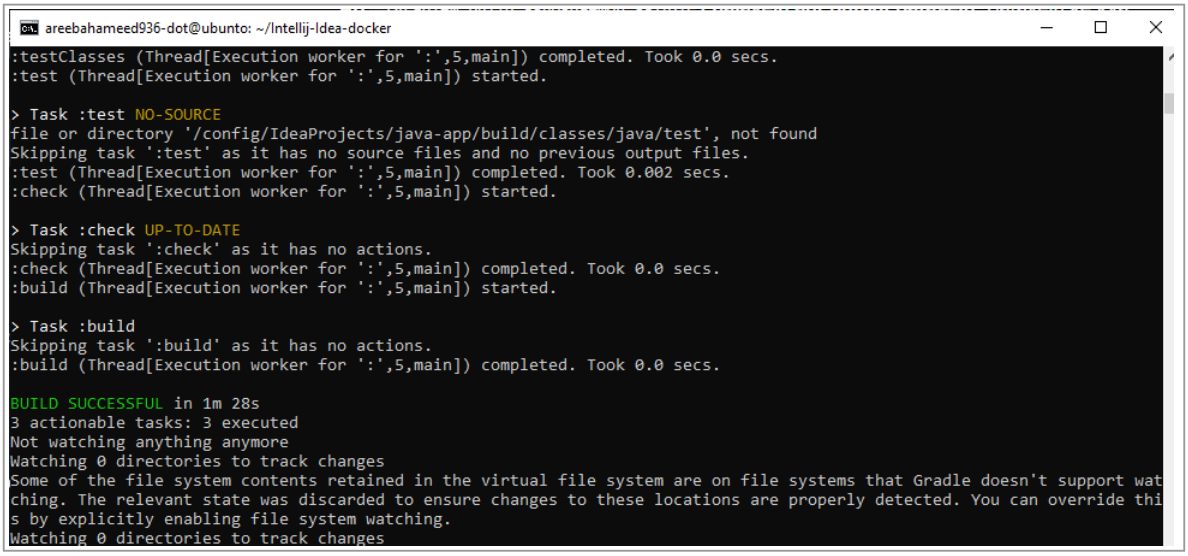
Confirm that the JVM information reports Java 16. Do not continue while it reports Java 21.

Build the project:

```
./gradlew build
```

The wrapper downloads the project's Gradle version on its first run. Wait for BUILD SUCCESSFUL.

Screenshot as `gradle_build.png`, showing the command and successful build.



Screenshot 15: gradle_build.png

If the build reports Unsupported class file major version 65, Gradle is still running on Java 21. Recheck both the Project SDK and Gradle JVM, reopen the terminal, confirm Java 16 with ./gradlew --version, and retry.

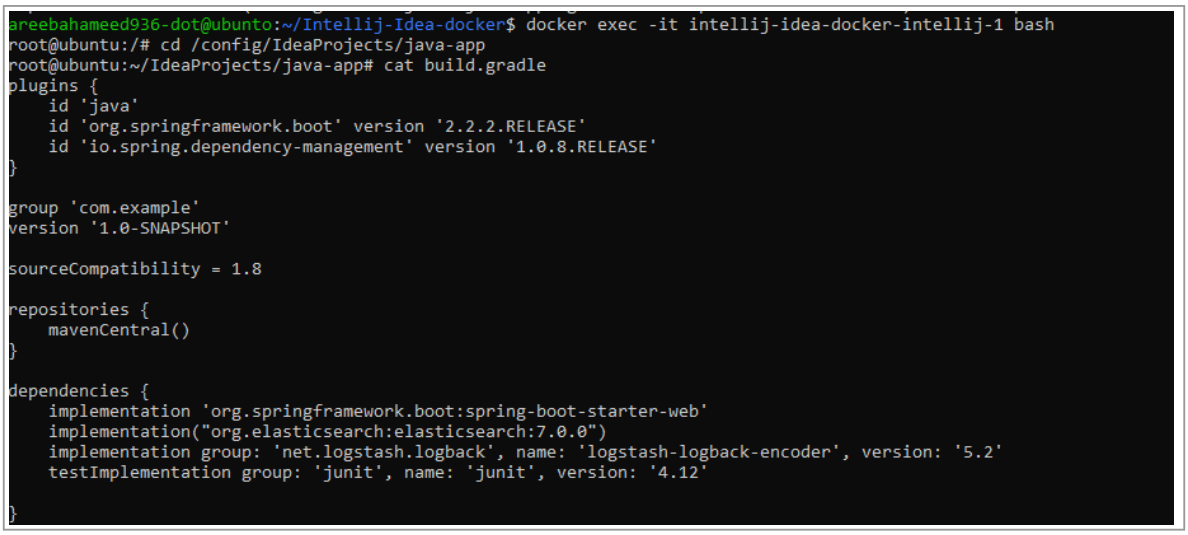
If the wrapper reports Permission denied, run chmod +x gradlew and retry ./gradlew build.

Open build.gradle and add this line inside the existing dependencies { ... } block:

```
implementation("org.elasticsearch:elasticsearch:7.0.0")
```

Keep the other dependencies. Save the file and click Sync Gradle Changes, or open the Gradle tool window and choose Sync All Gradle Projects. Wait for synchronization to finish, then locate Elasticsearch 7.0.0 under the project's External Libraries.

Screenshot as `gradle_elasticsearch_dependency.png`, showing the dependency in build.gradle and the synchronized Elasticsearch library. Keep your required terminal prompt visible.



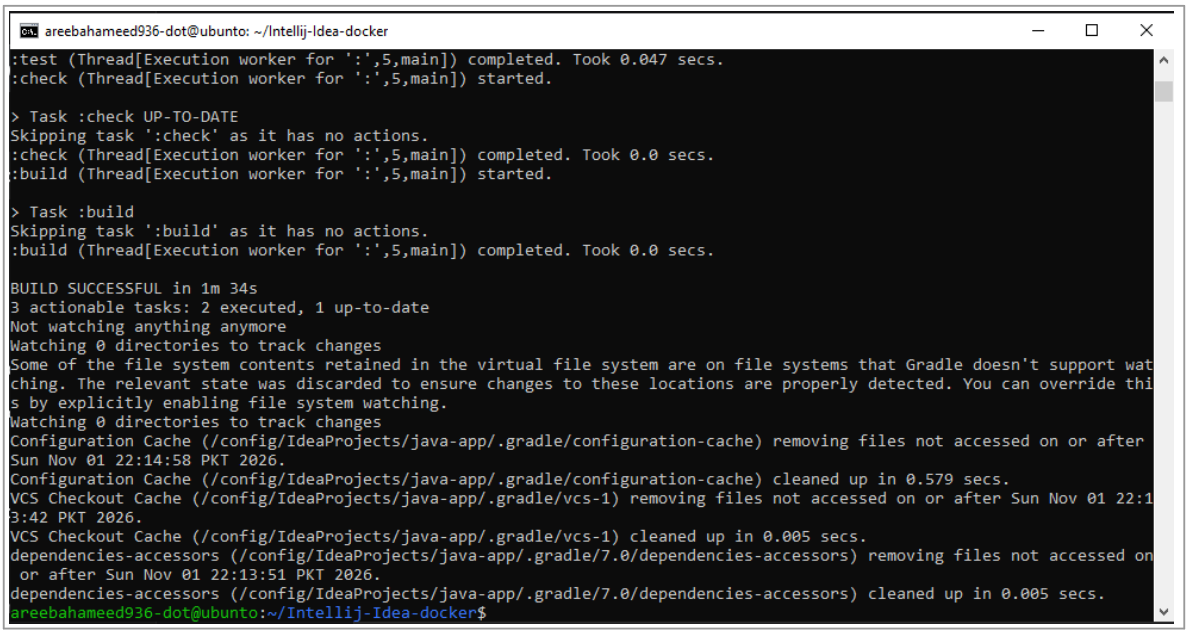
Screenshot 16: gradle_elasticsearch_dependency.png

Build the project again after the dependency change:

```
./gradlew build
```

Wait for BUILD SUCCESSFUL.

Screenshot as `gradle_rebuild.png`, showing the command and successful rebuild.

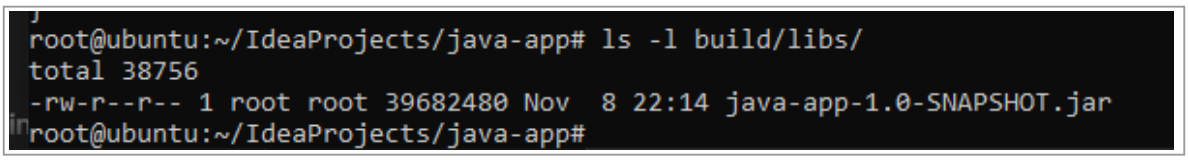


Screenshot 17: gradle_rebuild.png

In IntelliJ's Project pane, expand build → libs. Locate:

```
build/libs/java-app-1.0-SNAPSHOT.jar
```

Screenshot as `gradle_jar.png`, showing the output directory and exact JAR filename.



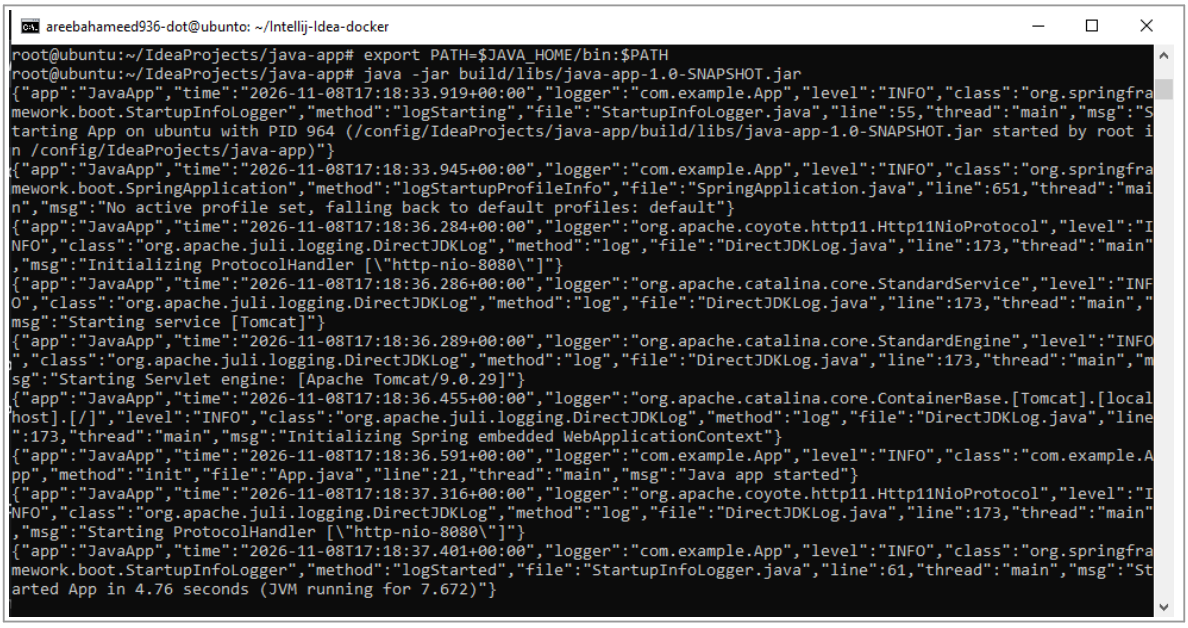
Screenshot 18: gradle_jar.png

Run the generated JAR from the project root:

```
java -jar build/libs/java-app-1.0-SNAPSHOT.jar
```

Wait for successful application startup.

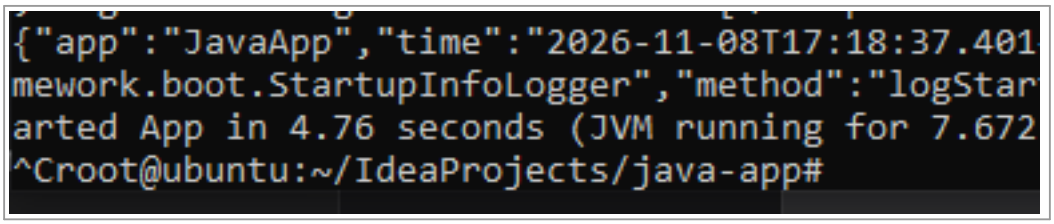
Screenshot as `gradle_running.png`, showing the command and startup output.



Screenshot 19: gradle_running.png

Press Ctrl+C and wait for the prompt to return. This also releases port 8080 before another application uses it.

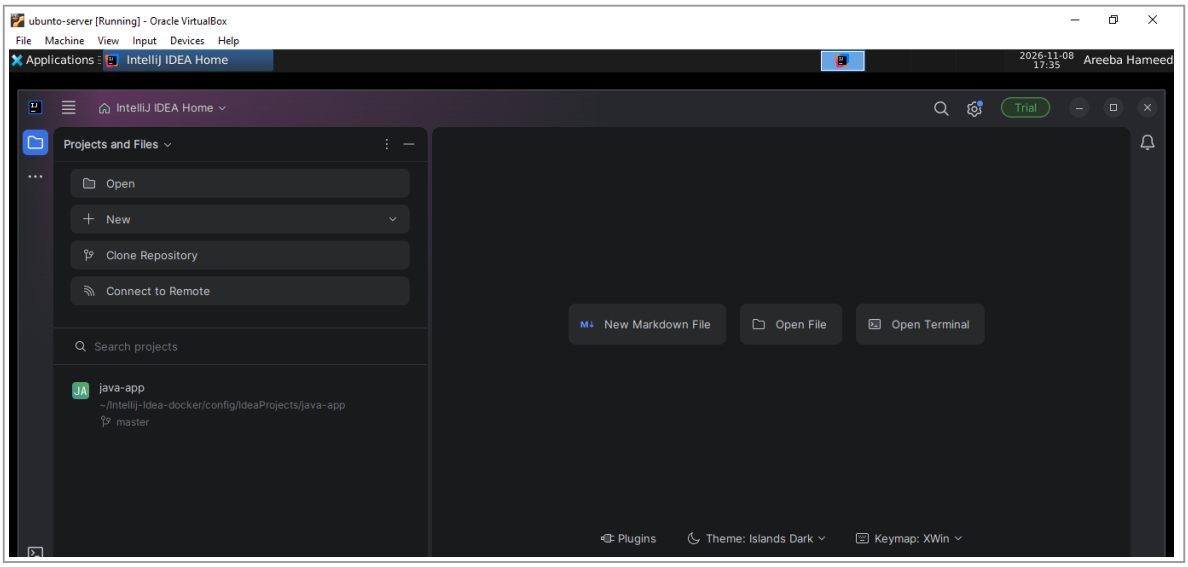
Screenshot as `gradle_stopped.png`, showing the shutdown and returned prompt.



Screenshot 20: gradle_stopped.png

Select File → Close Project.

Screenshot as `gradle_project_closed.png`, showing the Welcome screen beside your Ubuntu host terminal.



Screenshot 21: gradle_project_closed.png

Screenshot Required: `gradle_project_clone.png`, `gradle_build.png`, `gradle_elasticsearch_dependency.png`, `gradle_rebuild.png`, `gradle_jar.png`, `gradle_running.png`, `gradle_stopped.png`, `gradle_project_closed.png`

Summary: You configured the project and Gradle JVM to use JDK 16, added Elasticsearch 7.0.0 to build.gradle, synchronized the project and rebuilt it. The Gradle Wrapper created the executable JAR in build/libs, and Java ran it.

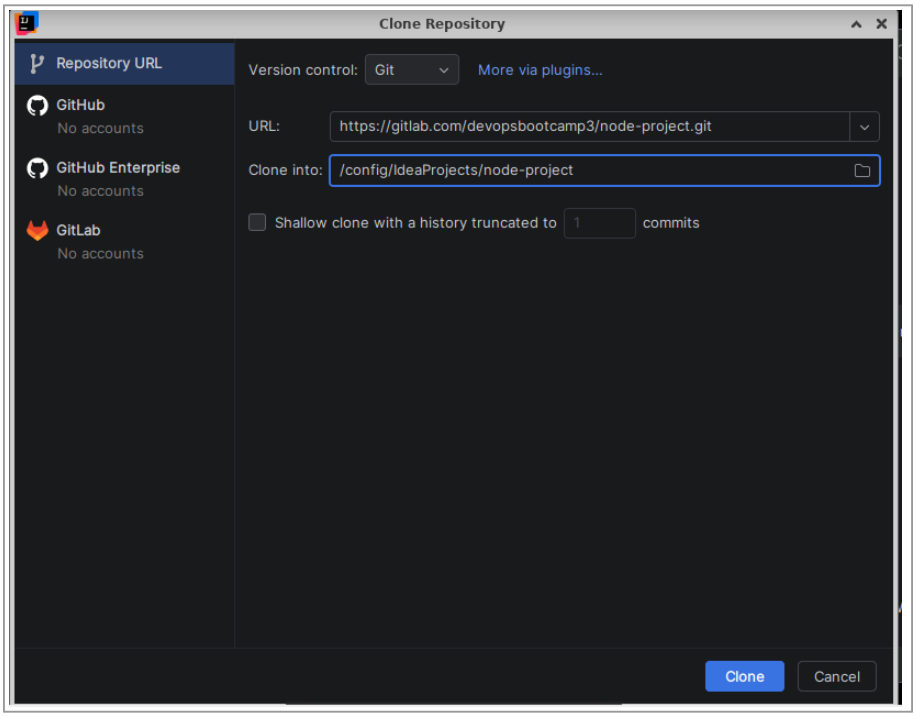
Task 4: Install and Run the Node.js Project

Use IntelliJ's Clone Repository / Get from Version Control action:

URL: https://gitlab.com/devopsbootcamp3/node-project.git

Directory: /config/IdeaProjects/node-project

Screenshot as `node_project_clone.png`, showing the URL and destination.



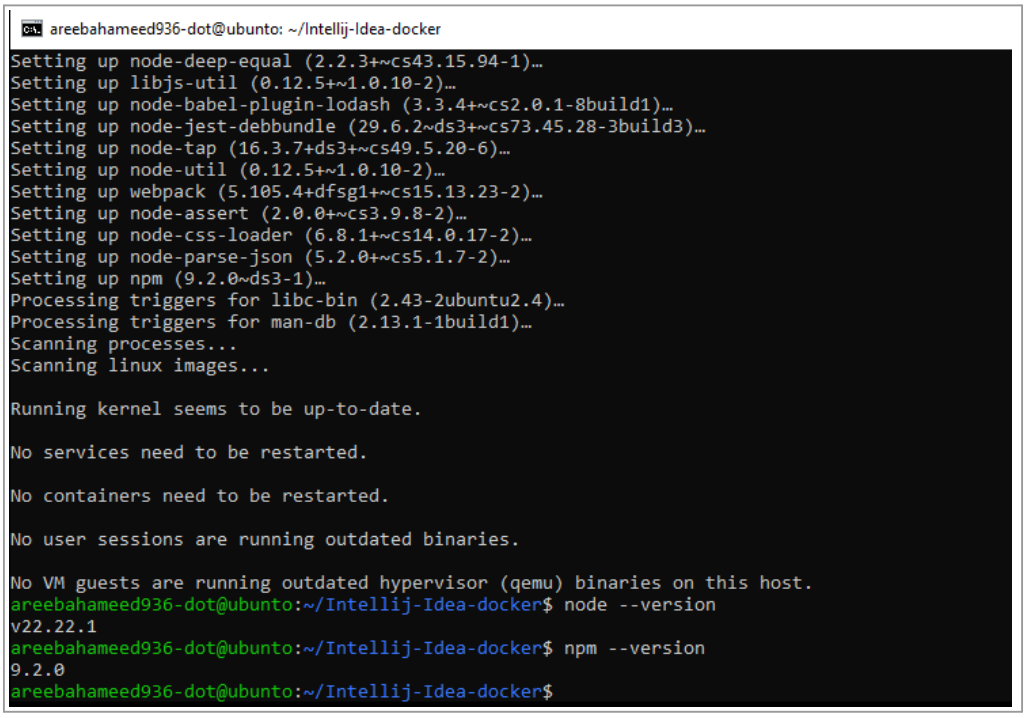
Screenshot 22: node_project_clone.png

Clone and open the project. Its application files are in the app subdirectory.

Open the terminal inside IntelliJ and install Node.js together with npm:

```
sudo apt update
sudo apt install -y nodejs npm
node --version
npm --version
```

Screenshot as `nodejs_installed.png`, showing both installed versions.



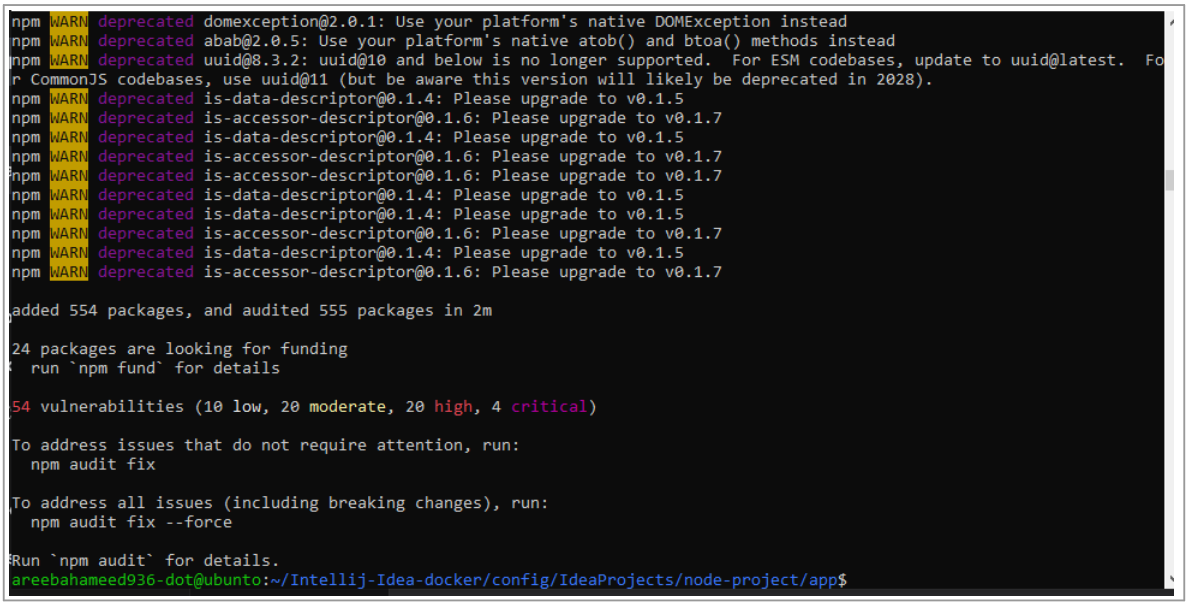
Screenshot 23: nodejs_installed.png

From the project root, enter app and install its dependencies:

```
cd app
npm install
```

The app/package.json file defines the dependencies and the start script.

Screenshot as `npm_install.png`, showing the command and completed installation.



Screenshot 24: npm_install.png

Start the application:

```
npm start
```

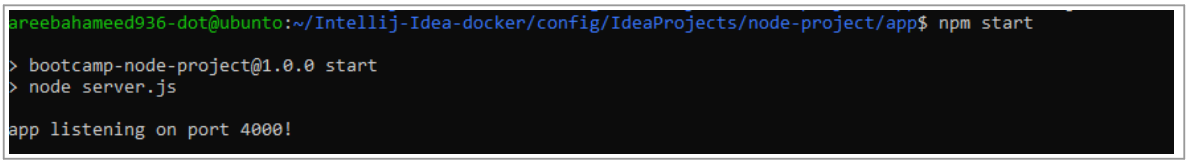
The supplied server.js initially listens on port 3000. Webtop also uses that port inside this container, so the initial start may fail with EADDRINUSE.

If port 3000 fails, press Ctrl+C if the process is still active. Open app/server.js in IntelliJ and change the listener and its message to:

```
app.listen(4000, function () {
  console.log("app listening on port 4000!");
});
```

Save the file and run npm start again from the app directory. Use port 4000 for the final browser check inside the container desktop. Keep Webtop running while changing the application port.

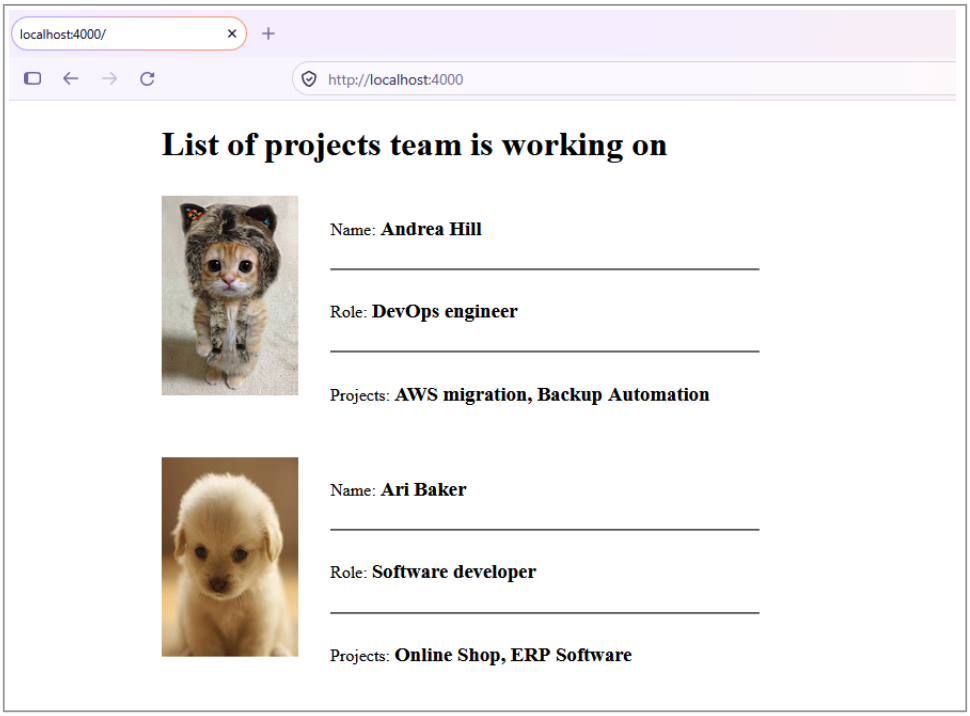
Screenshot as `node_running.png`, showing npm start and app listening on port 4000!.



Screenshot 25: node_running.png

Keep the application running. Open a browser inside the container's desktop from its applications menu and visit http://localhost:4000. Verify that the Node.js project page and its profile images are visible.

Screenshot as `node_browser.png`, showing the page, its URL and your IntelliJ terminal in a split-screen view inside the container desktop.



Screenshot 26: node_browser.png

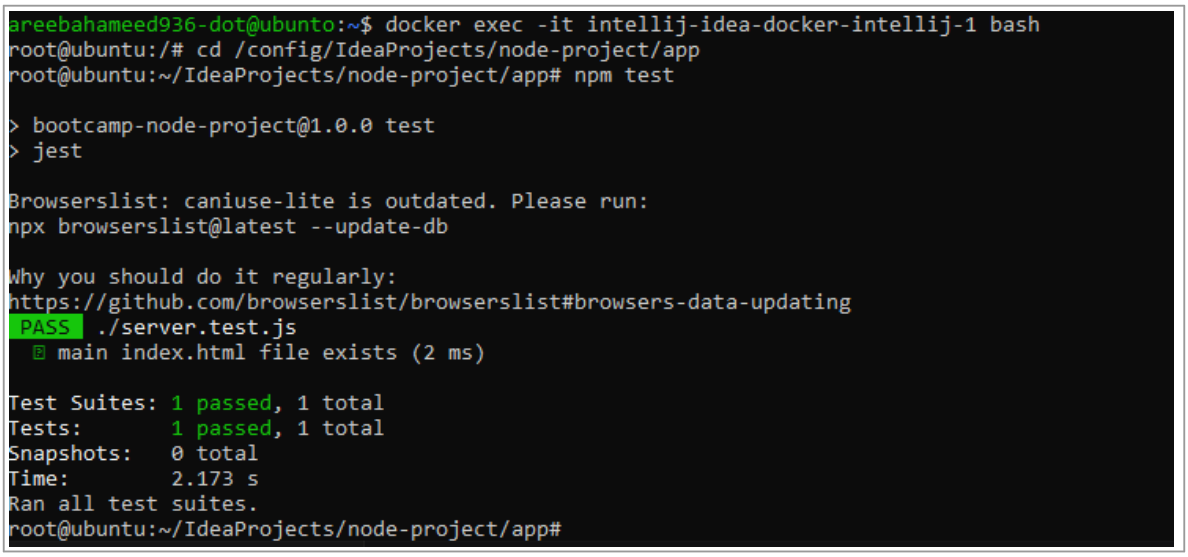
After saving the evidence, stop the Node.js application with Ctrl+C in IntelliJ's terminal.

Stay in the app directory and run the application's tests:

```
npm test
```

The project's test script runs Jest. The supplied test checks that index.html exists. Wait for the test result.

Screenshot as `npm_test.png`, showing the command and passing test summary.



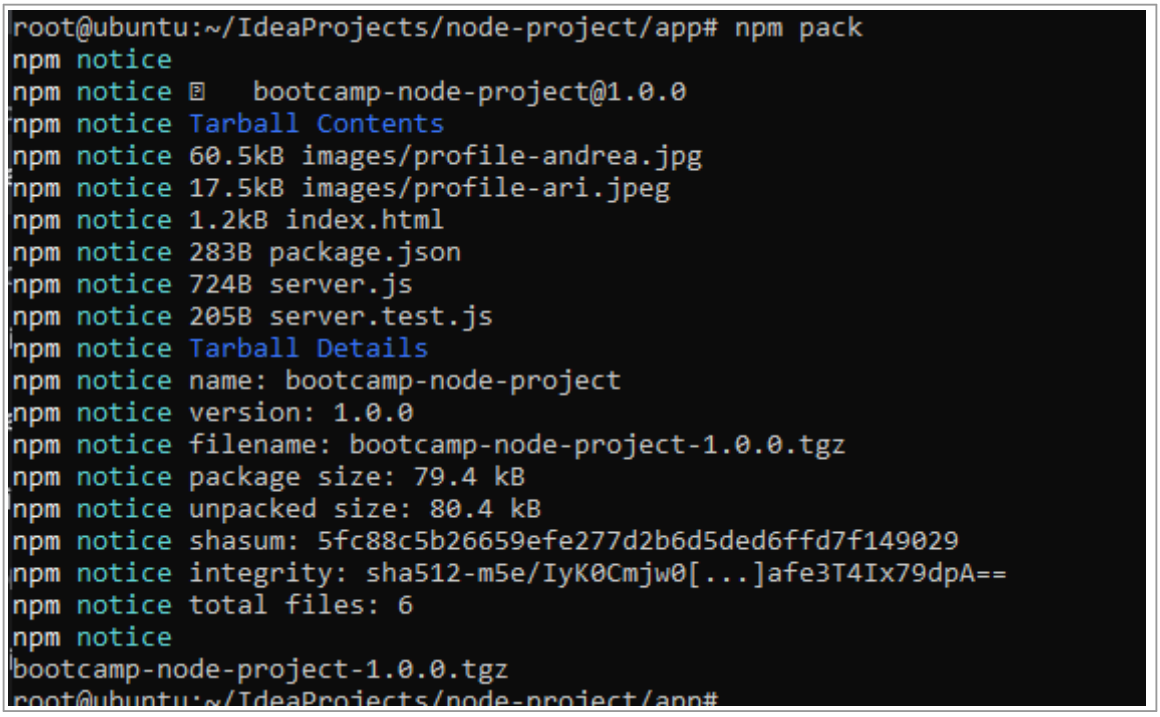
Screenshot 27: npm_test.png

Create the application's package artifact:

```
npm pack
```

npm uses the name and version in package.json to create bootcamp-node-project-1.0.0.tgz in the app directory. This tarball packages the application's files for distribution.

Screenshot as `npm_pack.png`, showing the command and generated .tgz filename.



Screenshot 28: npm_pack.png

Select File → Close Project to return to the Welcome screen. Keep this clone for Task 6.

Screenshot Required: `node_project_clone.png`, `nodejs_installed.png`, `npm_install.png`, `node_running.png`, `node_browser.png`, `npm_test.png`, `npm_pack.png`

Summary: npm installed dependencies, started the application, ran its Jest test and packaged the application as a .tgz artifact.

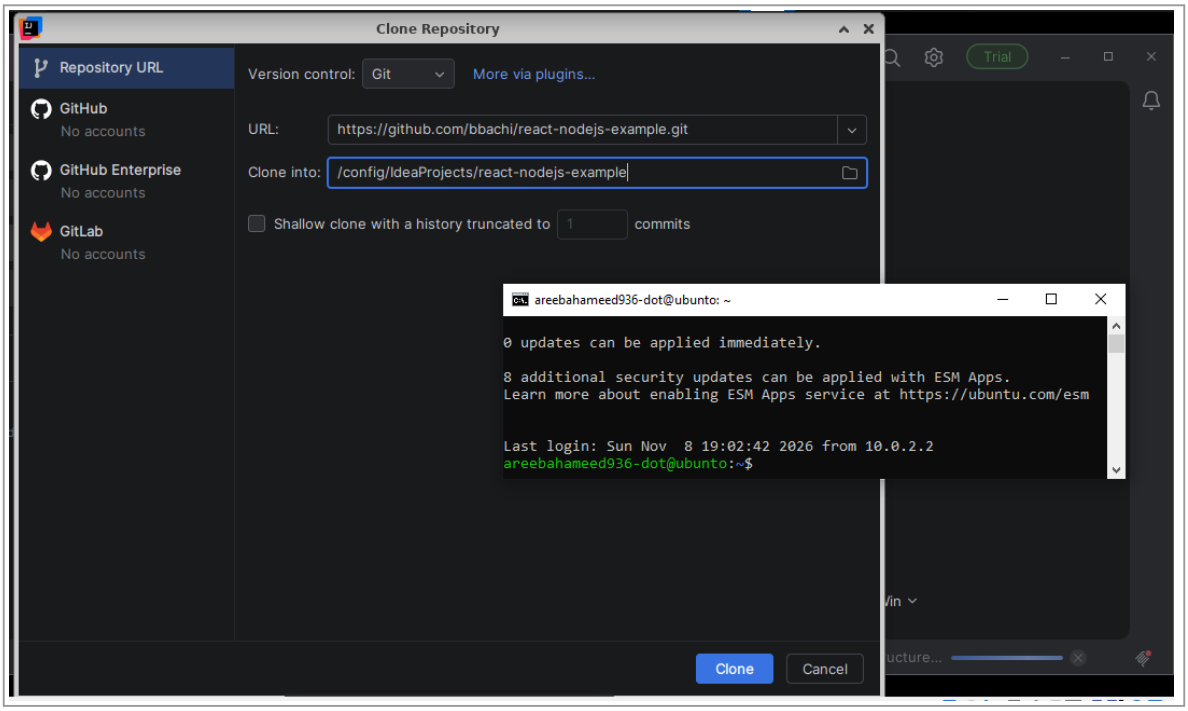
Task 5: Build the React and Node.js API Artifact

From IntelliJ's Welcome screen, use Clone Repository / Get from Version Control:

URL: https://github.com/bbachi/react-nodejs-example.git

Directory: /config/IdeaProjects/react-nodejs-example

Screenshot as `react_node_project_clone.png`, showing the repository URL and destination.



Screenshot 29: react_node_project_clone.png

Clone and open the project. This task builds the Node.js API in its api directory.

Open the terminal inside IntelliJ. From the project root, run:

```
cd api
npm install
```

After installation, refresh the Project pane if needed and expand api. Locate node_modules, which contains the installed dependencies.

Screenshot as `react_api_dependencies.png`, showing the completed npm install and the api/node_modules directory.

```
npm warn deprecated figgy-pudding@3.5.2: This module is no longer supported.
npm warn deprecated copy-concurrently@1.0.5: This package is no longer supported.
npm warn deprecated move-concurrently@1.0.1: This package is no longer supported.
npm warn deprecated lodash.clone@4.5.0: This package is deprecated. Use structuredClone instead.
npm warn deprecated fs-write-stream-atomic@1.0.10: This package is no longer supported.
npm warn deprecated clone-buffer@1.0.0: Package no longer supported. Contact Support at https://www.npmjs.com/support fo
r more info.
npm warn deprecated default-resolution@2.0.0: Package no longer supported. Contact Support at https://www.npmjs.com/supp
ort for more info.
npm warn deprecated urix@0.1.0: Please see https://github.com/lydell/urix#deprecated
npm warn deprecated source-map-url@0.4.1: See https://github.com/lydell/source-map-url#deprecated
npm warn deprecated resolve-url@0.2.1: https://github.com/lydell/resolve-url#deprecated
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache i
f you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerfu
l.
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabi
lities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorb
itant rates) by contacting i@izs.me
npm warn deprecated rimraf@3.0.2: Rimraf versions prior to v4 are no longer supported
npm warn deprecated rimraf@2.7.1: Rimraf versions prior to v4 are no longer supported
npm warn deprecated rimraf@2.7.1: Rimraf versions prior to v4 are no longer supported
npm warn deprecated rimraf@2.7.1: Rimraf versions prior to v4 are no longer supported
npm warn deprecated source-map-resolve@0.5.3: See https://github.com/lydell/source-map-resolve#deprecated

added 727 packages in 3m
root@ubuntu:~/IdeaProjects/react-nodejs-example/api#
root@ubuntu:~/IdeaProjects/react-nodejs-example/api# exit
exit
areebahameed936-dot@ubunto:~$
```

Screenshot 30: react_api_dependencies.png

In the same api directory, build the API artifact:

```
npm run build
```

The build script runs webpack. Its configuration writes the bundled application to api/server.bundle.js, replacing the existing bundle if one is already present.

If this older webpack sample reports ERR_OSSL_EVP_UNSUPPORTED on your Node.js version, retry this build with the compatibility option for that command:

```
NODE_OPTIONS=--openssl-legacy-provider npm run build
```

Screenshot as `react_api_build.png`, showing the successful build command/output and server.bundle.js in the api directory.

```
          Asset     Size  Chunks             Chunk Names
server.bundle.js  606 KiB       0  [emitted]  main
Entrypoint main = server.bundle.js
 [1] external "path" 42 bytes {0} [built]
 [5] external "fs" 42 bytes {0} [built]
 [7] external "buffer" 42 bytes {0} [built]
[10] external "stream" 42 bytes {0} [built]
[16] external "util" 42 bytes {0} [built]
[24] external "http" 42 bytes {0} [built]
[38] external "net" 42 bytes {0} [built]
[40] external "events" 42 bytes {0} [built]
[41] external "zlib" 42 bytes {0} [built]
[45] external "async_hooks" 42 bytes {0} [optional] [built]
[57] external "querystring" 42 bytes {0} [built]
[58] external "url" 42 bytes {0} [built]
[65] external "crypto" 42 bytes {0} [built]
[69] ./server.js 990 bytes {0} [built]
[80] external "tty" 42 bytes {0} [built]
    + 145 hidden modules

WARNING in ./node_modules/express/lib/view.js 81:13-25
Critical dependency: the request of a dependency is an expression
 @ ./node_modules/express/lib/application.js
 @ ./node_modules/express/lib/express.js
 @ ./node_modules/express/index.js
 @ ./server.js
root@ubuntu:~/IdeaProjects/react-nodejs-example/api# exit
exit
areebahameed936-dot@ubunto:~$
```

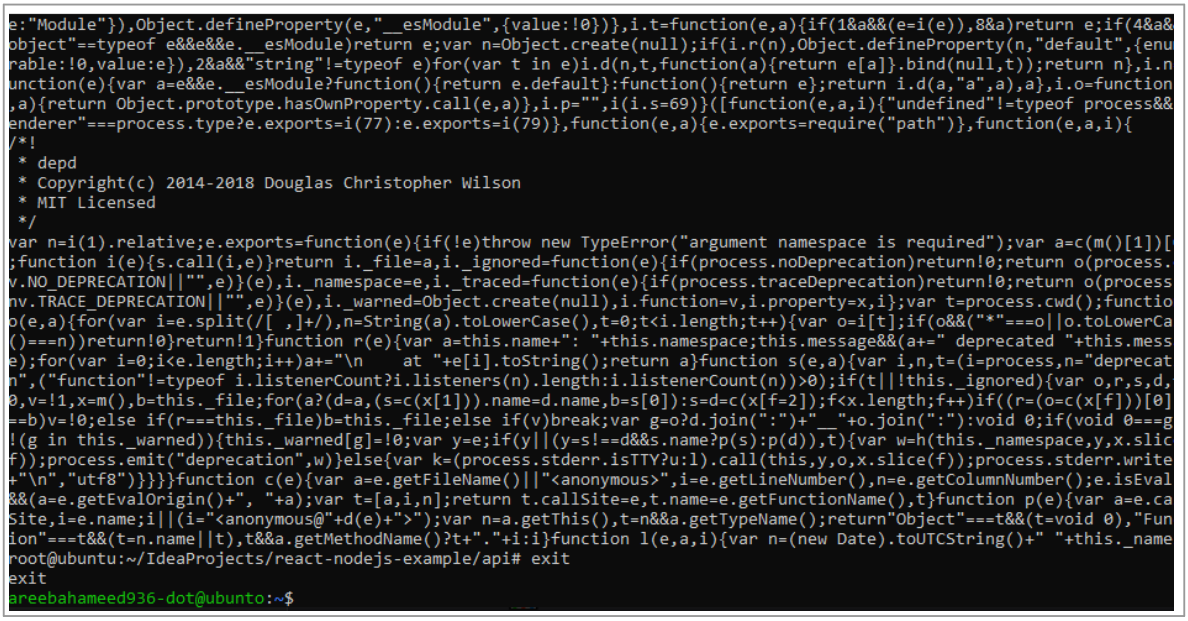
Screenshot 31: react_api_build.png

View the generated bundle:

```
cat server.bundle.js | more
```

Press Space to move through its contents.

Screenshot as `react_api_bundle.png`, showing the viewing command and bundled JavaScript. Use a split terminal view to keep your required identity visible while the pager is open.



Screenshot 32: react_api_bundle.png

Press q to quit the viewer and return to the terminal. Select File → Close Project.

Screenshot Required: `react_node_project_clone.png`, `react_api_dependencies.png`, `react_api_build.png`, `react_api_bundle.png`

Summary: You installed the API dependencies, used webpack to generate server.bundle.js and inspected the bundled artifact.

Task 6: Build a Docker Image for the Node.js Project

In IntelliJ, choose Open from the Welcome screen, or File → Open, and reopen the existing /config/IdeaProjects/node-project clone from Task 4:

Repository: https://gitlab.com/devopsbootcamp3/node-project.git

Open IntelliJ's terminal and move to the repository root:

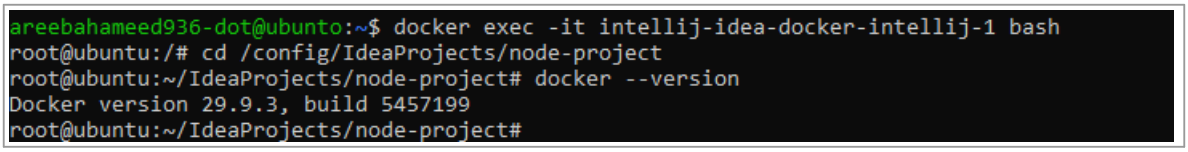
```
cd /config/IdeaProjects/node-project
```

The Dockerfile is in this directory, alongside the app folder. Run all commands in this task from this root directory.

Check that the Docker command is installed:

```
docker --version
```

Screenshot as `docker_installed.png`, showing the command and installed version.



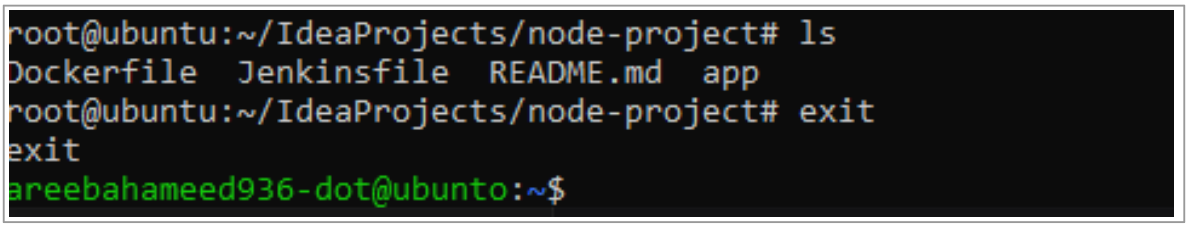
Screenshot 33: docker_installed.png

The current IntelliJ setup repository shares the Ubuntu host's Docker socket with the desktop container. The following sudo docker commands use that host's Docker Engine. If you are using an older environment and see a missing /var/run/docker.sock error, apply the existing-installation update in the IntelliJ repository instructions from your Ubuntu host before continuing.

List the repository files and locate Dockerfile:

```
ls
```

Screenshot as `dockerfile_exists.png`, showing the command and Dockerfile in the output.

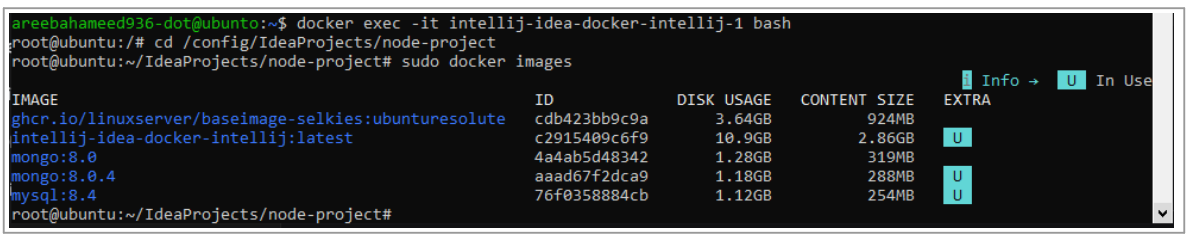


Screenshot 34: dockerfile_exists.png

View the existing Docker images before building:

```
sudo docker images
```

Screenshot as `docker_images_before.png`, showing the command and current image list.



Screenshot 35: docker_images_before.png

Build the application image:

```
sudo docker build -t lab5:1.0 .
```

The final . uses the current repository root as the build context. Docker reads its Dockerfile, copies the app directory, installs dependencies and runs the supplied npm test during the image build. Wait for the build to finish successfully.

Screenshot as `docker_build.png`, showing the build command and successful image build/tag output.



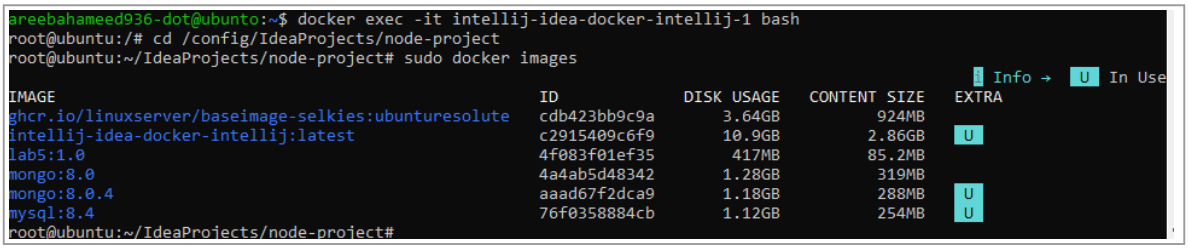
Screenshot 36: docker_build.png

List the images again:

```
sudo docker images
```

Find lab5 in the REPOSITORY column and 1.0 in the TAG column. Together they identify lab5:1.0.

Screenshot as `docker_images_after.png`, showing the command, image repository, tag and image ID.



Screenshot 37: docker_images_after.png

Screenshot Required: `docker_installed.png`, `dockerfile_exists.png`, `docker_images_before.png`, `docker_build.png`, `docker_images_after.png`

Summary: You built the existing Node.js project into the Docker image lab5:1.0 and verified it in the host's image list.